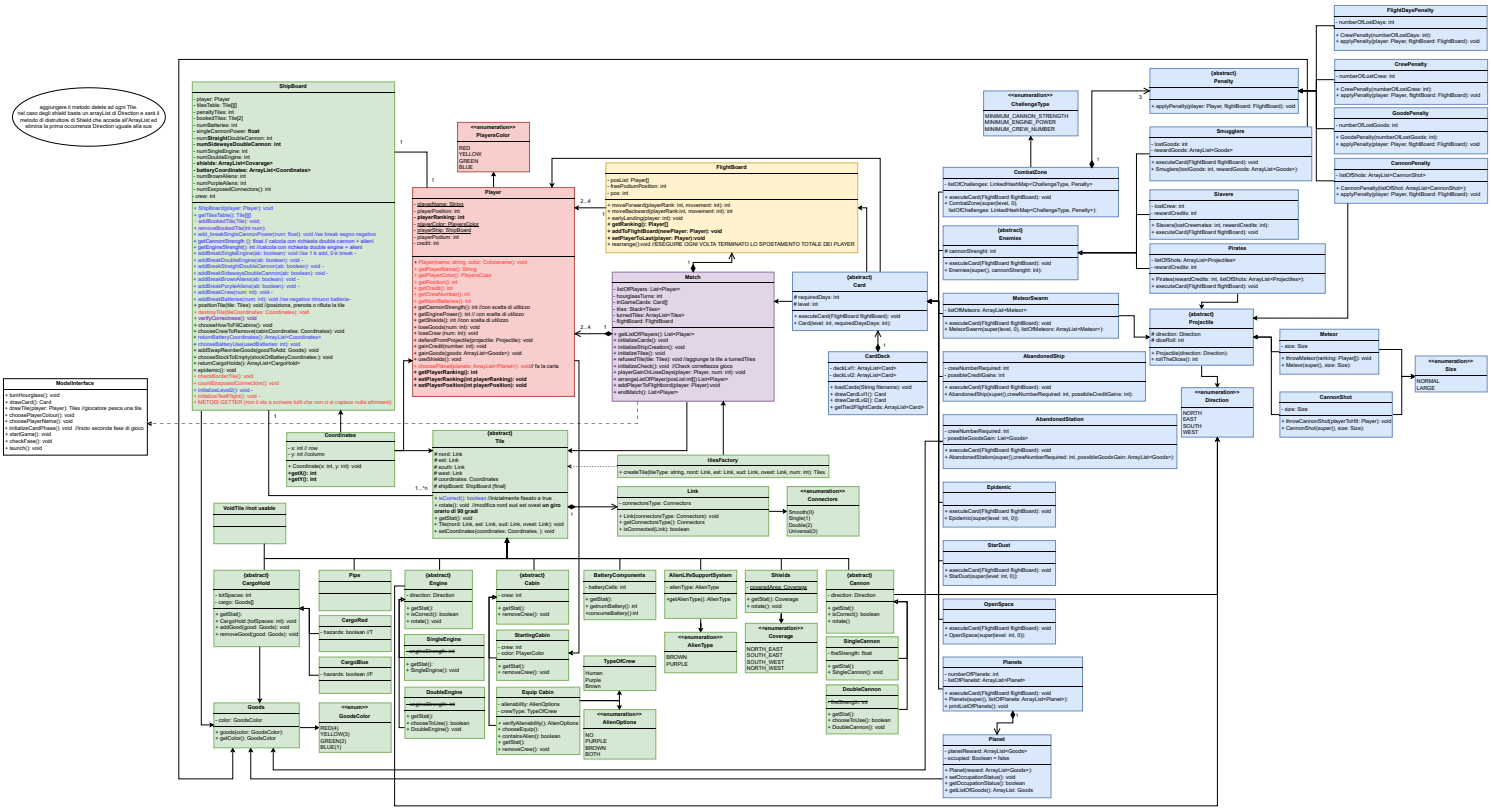
aggiungere il metodo delete ad ogni Tile
nel caso degli shield basta un arrayList di Direction e cardi il metodo di distruttore di Shield che accede all'ArrayList ed elimina la prima occorrenza Direction uguale alla sua
ShipBoard
- player: Player
- tileTiles: Tile[][]
- penalTiles: int
- bookedTiles: Tile[]
- numBatteries: int
- singleCannonPower: float
- numStraightDoubleCannon: int
- numSidewaysDoubleCannon: int
- numSingleEngine: int
- numDoubleEngine: int
- shields: ArrayList<Coverage>
- batteryCoordinates: ArrayList<Coordinates>
- numBrownAliens: int
- numPurpleAliens: int
- numExposedConnectors(): int
+ crew: int
+ ShipBoard(player: Player): void
+ getTilesTiles(): Tile[][]
- addBookedTile(Tile): void;
+ removeBookedTile(int num):
+ add_breakSingleCannonPower(num: float): void //se break segno negativo
+ getCannonStrength (): float // calcola con richiesta double cannon + alieni
+ getEngineStrength(): int //calcola con richiesta double engine + alieni
+ addBreakSingleEngine(ab: boolean): void //se 1 è add, 0 è break -
+ addBreakSingleCannon(ab: boolean): void -
+ addBreakStraightDoubleCannon(ab: boolean): void -
+ addBreakSidewaysDoubleCannon(ab: boolean): void -
+ addBreakBrownAliens(ab: boolean): void -
+ addBreakPurpleAliens(ab: boolean): void -
+ addBreakCrew(num: int): void -
+ addBreakBatteries(num: int): void //se negativo rimuovi batteria-
+ positionTile(Tile: Tile): void //posiziona, prendo o rifiuta la tile
- destroyTile(tileCoordinates: Coordinates): void
+ verifyCorrectness(): void
+ chooseHowToFillCabins(): void
+ chooseCrewToRemove(cabinCoordinates: Coordinates): void
+ returnBatteryCoordinates(): ArrayList<Coordinates>
+ removeBattery(coordinates: Coordinates): int -void
+ addTileInOrder(tileToAdd: Goods): void
+ chooseStockToEmpty(stockToBatteryCoordinates:): void
+ returnCargoHolds(): ArrayList<CargoHold>
+ epidemic(): void
- checkBorderTile(): void
- countExposedConnectors(): void
+ initializeLevel2(): void -
+ initializeTestFlight(): void -
// METODI GETTER (non li sto a scrivere tutti che non ci si capisce nulla altrimenti)
<<enumeration>>
PlayersColor
RED
YELLOW
GREEN
BLUE
Player
- playerName: String
- playerPosition: int
- playerRanking: int
- playerColor: PlayersColor
- playerShip: ShipBoard
- playerPodium: int
- credit: int
+ Player(name: string, color: Colorenums): void
+ getPlayerName(): String
+ getPlayerColor(): PlayersColor
+ getPosition(): int
+ getCredit(): int
+ getCrew(number): int
- getNumBatteries(): int
+ getCannonStrength(): int //con scelta di utilizzo
+ getEnginePower(): int // con scelta di utilizzo
+ getShields(): int //con scelta di utilizzo
+ loseGoods(num: int): void
+ loseCrew (num: int): void
+ defendFromProjectile(projectile: Projectile): void
+ gainCredit(number: int): void
+ gainGoods(goods: ArrayList<Goods>): void
+ useShields(): void
+ controllaPenalità(): ArrayList<Planet>: voidil fa la carta
+ getPlayerRanking(): int
+ setPlayerRanking(int playerRanking): void
+ setPlayerPosition(int playerPosition): void
FlightBoard
- posList: Player[]
- freePodiumPosition: int
- pos: int
+ moveForward(playerRack: int, movement: int): int
+ moveBackward(playerRack:int, movement: int): int
+ manyLanding(player: int): void
+ getRanking(): Player[]
+ addToFlightBoard(newPlayer: Player): void
+ setPlayerToLast(player: Player):void
+ rearrange():void //ESEGUIRE OGNI VOLTA TERMINATO LO SPOSTAMENTO TOTALE DEI PLAYER
Match
- listOfPlayers: List<Player>
- hourglassTurns: int
- inGameCards: Card[]
- lobby: Stack<Player>
- tiles: ArrayList<Tile>
- flightBoard: FlightBoard
+ getListOfPlayers(): List<Player>
+ initializeCards(): void
+ initializeShipCreation(): void
+ initializeTiles(): void
+ refusedTile(Tile: Tile): void //aggiunge la tile a turnedTiles
+ initializeCheck(): void //Check correttezza gioco
+ playerGainOrLoseDays(player: Player, num: int): void
+ arrangeListOfPlayer(goodList:int[]):List<Player>
+ addPlayerToFlightBoard(player: Player):void
+ endMatch(): List<Player>
(abstract)
Card
requiredDays: int
level: int
+ executeCard(FlightBoard flightBoard): void
+ Card(level: int, requiredDaysDays: int):
CardDeck
- deckLvl1: ArrayList<Card>
- deckLvl2: ArrayList<Card>
+ loadCards(String filename): void
+ drawCard(Lvl1): Card
+ drawCard(Lvl2) : Card
+ getTwoFlightCards: ArrayList<Card>
Coordinates
- x: int // row
- y: int //column
+ Coordinate(x: int, y: int): void
+ getX(): int
+ getY(): int
(abstract)
Tile
nord: Link
est: Link
south: Link
west: Link
coordinates: Coordinates
shipBoard: ShipBoard (final)
+ isConnect(): boolean //inizialmente fissato a true
+ rotate(): void //tessellina nord sud est ovest un giro orario di 90 gradi
+ getSlot(): void
+ Tile(nord: Link, est: Link, sud: Link, ovest: Link): void
+ setCoordinates(coordinates: Coordinates,): void
tilesFactory
+ createTile(tileType: string, nord: Link, est: Link, sud: Link, ovest: Link, num: int): Tiles
Link
- connectionsType: Connectors
+ Link(connectionsType: Connectors): void
+ getConnectionsType(): Connectors
+ isConnected(Link): boolean
<<enumeration>>
Connectors
Smooth(0)
Single(1)
Double(2)
Universal(3)
VoidTile //not usable
(abstract)
CargoHold
- totSpaces: int
- cargo: Goods[]
+ getSlot():
+ CargoHold (totSpaces: int): void
+ addGood(good: Goods): void
+ removeGood(good: Goods): void
Pipe
CargoRed
- hazards: boolean rT
CargoBlue
- hazards: boolean rF
Goods
- color: GoodsColor
+ goods(color: GoodsColor):
+ getColor(): GoodsColor
<<enum>>
GoodsColor
RED(4)
YELLOW(3)
GREEN(2)
BLUE(1)
(abstract)
Engine
- direction: Direction
+ getSlot()
+ isCorrect(): boolean
+ rotate(): void
SingleEngine
- engineStrength: int
+ getSlot()
+ SingleEngine(): void
DoubleEngine
- engineStrength: int
+ getSlot()
+ chooseToUse(): boolean
+ DoubleEngine(): void
(abstract)
Cabin
- crew: int
+ getSlot()
+ removeCrew(): void
StartingCabin
- crew: int
- color: PlayerColor
+ getSlot()
+ removeCrew(): void
Equip Cabin
- alienability: AlienOptions
- crewType: TypeOfCrew
+ verifyAlienability(): AlienOptions
+ chooseEquip()
+ containsAlien(): boolean
+ getSlot()
+ removeCrew(): void
BatteryComponents
- batteryCells: int
+ getSlot():
+ getnumBattery(): int
+consumeBattery():int
AlienLifeSupportSystem
- alienType: AlienType
+getAlienType(): AlienType
<<enumeration>>
AlienType
BROWN
PURPLE
TypeOfCrew
Human
Purple
Brown
<<enumeration>>
AlienOptions
NO
PURPLE
BROWN
BOTH
Shields
- coverageMask: Coverage
+ getSlot(): Coverage
+ rotate(): void
<<enumeration>>
Coverage
NORTH_EAST
SOUTH_EAST
SOUTH_WEST
NORTH_WEST
(abstract)
Cannon
- direction: Direction
+ getSlot()
+ isCorrect(): boolean
+ rotate()
SingleCannon
- fireStrength: float
+ getSlot()
+ SingleCannon(): void
DoubleCannon
- fireStrength: int
+ getSlot()
+ chooseToUse(): boolean
+ DoubleCannon(): void
ModelInterface
+ turnYourgoes(): void
+ drawCard(): Card
+ drawTile(player: Player): Tiles //giocatore pesca una tile
+ choosePlayerColour(): void
+ chooseYourName(): void
+ chooseNumPlayers(): void //inizio seconda fase di gioco
+ startGame(): void
+ checkFase(): void
+ launch(): void
<<enumeration>>
ChallengeType
MINIMUM_CANNON_STRENGTH
MINIMUM_ENGINE_POWER
MINIMUM_CREW_NUMBER
CombatZone
- listOfChallenges: LinkedHashMap<ChallengeType, Penalty>
+ executeCard(FlightBoard flightBoard): void
+ CombatZone(super(level, 0),
- listOfChallenges: LinkedHashMap<ChallengeType, Penalty>)
(abstract)
Enemies
cannonStrenght: int
+ executeCard(FlightBoard flightBoard): void
+ Enemies(super(), cannonStrenght: int):
MeteorSwarm
- listOfMeteors: ArrayList<Meteor>
+ executeCard(FlightBoard flightBoard): void
+ MeteorSwarm(super(level, 0), listOfMeteors: ArrayList<Meteor>)
AbandonedShip
- crewNumberRequired: int
- possibleCreditGains: int
+ executeCard(FlightBoard flightBoard): void
+ AbandonedShip(super(),crewNumberRequired: int, possibleCreditGains: int):
AbandonedStation
- crewNumberRequired: int
- possibleGoodsGain: List<Goods>
+ executeCard(FlightBoard flightBoard): void
+ AbandonedStation(super(),crewNumberRequired: int, possibleGoodsGain: ArrayList<Goods>):
Epidemic
+ executeCard(FlightBoard flightBoard): void
+ Epidemic(super(level: int, 0));
StarDust
+ executeCard(FlightBoard flightBoard): void
+ StarDust(super(level: int, 0));
OpenSpace
+ executeCard(FlightBoard flightBoard): void
+ OpenSpace(super(level: int, 0));
Planets
- numberOfPlanets: int
- listOfPlanets: ArrayList<Planet>
+ executeCard(FlightBoard flightBoard): void
+ Planets(super(), listOfPlanets: ArrayList<Planet>)
+ printListOfPlanets(): void
Planet
- planetReward: ArrayList<Goods>
- occupied: Boolean = false
+ Planet(reward: ArrayList<Goods>)
+ setOccupationStatus(): void
+ getOccupationStatus(): boolean
+ getListOfGoods(): ArrayList: Goods
(abstract)
Penalty
+ applyPenalty(player: Player, flightBoard: FlightBoard): void
Smugglers
- lostGoods: int
- rewardGoods: ArrayList<Goods>
+ executeCard(FlightBoard flightBoard): void
+ Smugglers(lostGoods: int, rewardGoods: ArrayList<Goods>):
Slavers
- lostCrew: int
- rewardCredits: int
+ Slavers(lostCrewmates: int, rewardCredits: int):
+ executeCard(FlightBoard flightBoard): void
Pirates
- listOfShots: ArrayList<Projectiles>
- rewardCredits: int
+ Pirates(rewardCredits: int, listOfShots: ArrayList<Projectiles>)
+ executeCard(FlightBoard flightBoard): void
(abstract)
Projectile
direction: Direction
diceRoll: int
+ Projectile(direction: Direction):
+ rollTheDices(): int
<<enumeration>>
Direction
NORTH
EAST
SOUTH
WEST
Meteor
- size: Size
+ throwMeteor(ranking: Player[]): void
+ Meteor(super(), size: Size)
CannonShot
- size: Size
+ throwCannonShot(playerToHit: Player): void
+ CannonShot(super(), size: Size)
<<enumeration>>
Size
NORMAL
LARGE
FlightDaysPenalty
- numberOfLostDays: int
+ CrewPenalty(numberOfLostDays: int):
+ applyPenalty(player: Player, flightBoard: FlightBoard): void
CrewPenalty
- numberOfLostCrew: int
+ CrewPenalty(numberOfLostCrew: int):
+ applyPenalty(player: Player, flightBoard: FlightBoard): void
GoodsPenalty
- numberOfLostGoods: int
+ GoodsPenalty(numberOfLostGoods: int):
+ applyPenalty(player: Player, flightBoard: FlightBoard): void
CannonPenalty
- listOfShots: ArrayList<CannonShot>
+ CannonPenalty(listOfShot: ArrayList<CannonShot>):
+ applyPenalty(player: Player, flightBoard: FlightBoard): void